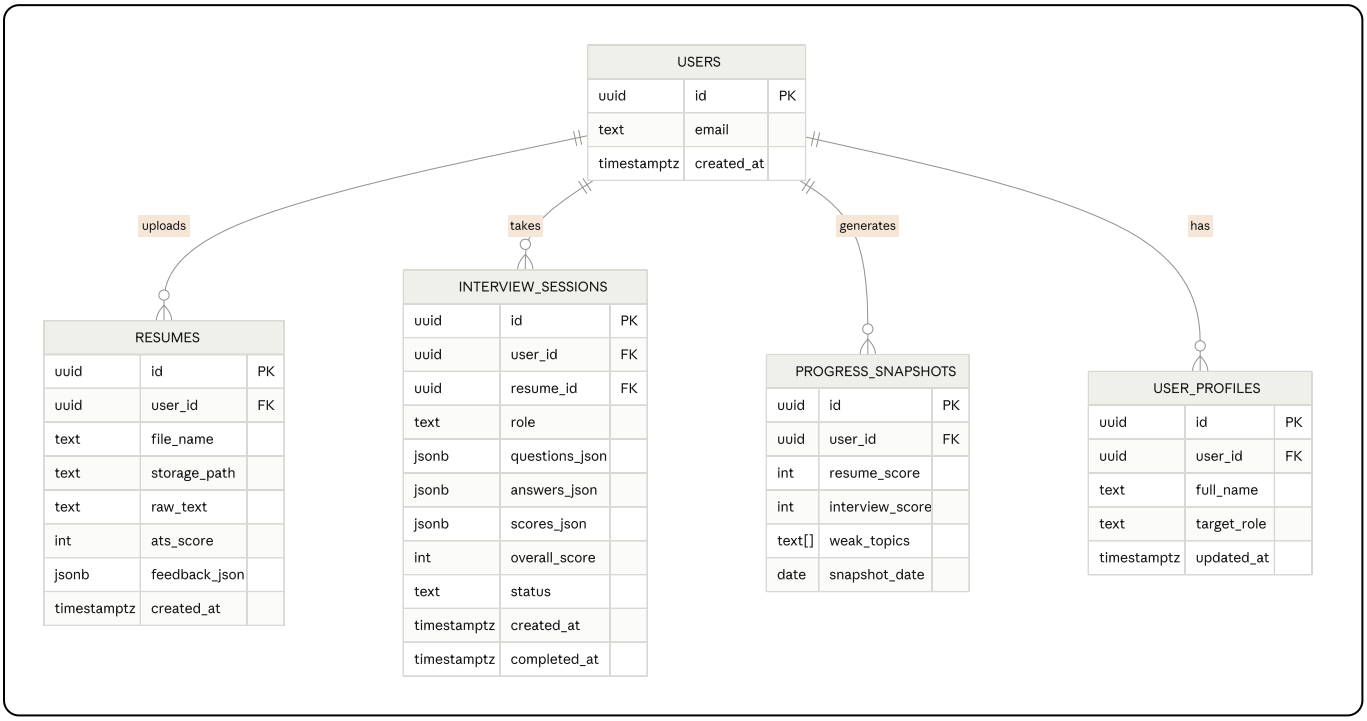
SCHEMA.md — PrepGenie Database Schema

Database: **PostgreSQL** (via Supabase). `auth.users` is managed by Supabase Auth; all app tables reference it via `user_id`.

---

1. Entity Relationship Diagram (Mermaid)



---

2. Table Definitions

2.1 `auth.users` (Managed by Supabase — not created manually)

Provided automatically by Supabase Auth. Referenced by all app tables via `user_id uuid REFERENCES auth.users(id)`.

2.2 `user_profiles`

Stores app-specific profile data not covered by Supabase Auth.

```sql
create table user_profiles (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    full_name text,
    target_role text check (target_role in ('SDE Intern','Data Analyst','Core Engineer
    updated_at timestamptz default now(),
    unique(user_id)
);
```

Constraints: `user_id` unique (one profile per user). `target_role` restricted to known enum values used across the app.

2.3 `resumes`

Stores each uploaded resume and its AI-generated analysis.

```sql
create table resumes (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    file_name text not null,
    storage_path text not null,
    raw_text text not null,
    ats_score int check (ats_score between 0 and 100),
    feedback_json jsonb,
    created_at timestamptz default now()
);

create index idx_resumes_user_id on resumes(user_id);
create index idx_resumes_created_at on resumes(created_at desc);
```

Constraints: `ats_score` bounded 0–100. `raw_text` required (empty extraction should be rejected at the application layer before insert). Indexed on `user_id` and `created_at` for fast dashboard history queries.

2.4 `interview_sessions`

Stores each mock interview attempt, including generated questions, user answers, and AI scores.

```sql
create table interview_sessions (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    resume_id uuid references resumes(id) on delete set null,
    role text not null check (role in ('SDE Intern','Data Analyst','Core Engineering',
    questions_json jsonb not null,
    answers_json jsonb default '[]'::jsonb,
    scores_json jsonb,
    overall_score int check (overall_score between 0 and 100),
    status text not null default 'in_progress' check (status in ('in_progress','comple
    created_at timestamptz default now(),
    completed_at timestamptz
);

create index idx_sessions_user_id on interview_sessions(user_id);
create index idx_sessions_status on interview_sessions(status);
```

Constraints: `role` restricted to the 4 supported tracks (PRD §5.3). `status` drives UI (resume-in-progress vs. show results). `resume_id` uses `on delete set null` so deleting a resume doesn't destroy interview history.

2.5 `progress_snapshots`

Denormalized rollup table powering the dashboard charts without expensive joins on every page load.

```sql
create table progress_snapshots (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references auth.users(id) on delete cascade,
    resume_score int,
    interview_score int,
    weak_topics text[],
    snapshot_date date not null default current_date,
    created_at timestamptz default now()
);

create index idx_snapshots_user_date on progress_snapshots(user_id, snapshot_date);
```

Constraints: One row inserted per completed resume analysis or interview session (append-only log, not upsert) — this preserves full history for the line/radar charts.

---

3. Row-Level Security (RLS) Policy Summary

All tables have RLS **enabled**, ensuring users can only access their own data — critical since this is a free Supabase project accessible via anon key on the frontend.

```sql
```

```
alter table user_profiles enable row level security;
alter table resumes enable row level security;
alter table interview_sessions enable row level security;
alter table progress_snapshots enable row level security;

create policy "Users manage own profile" on user_profiles
    for all using (auth.uid() = user_id);

create policy "Users manage own resumes" on resumes
    for all using (auth.uid() = user_id);

create policy "Users manage own sessions" on interview_sessions
    for all using (auth.uid() = user_id);

create policy "Users manage own snapshots" on progress_snapshots
    for all using (auth.uid() = user_id);
```

---

4. Supabase Storage Bucket

```
Bucket: resumes-bucket (private)
Path convention: {user_id}/{resume_id}-{original_filename}
Access policy: authenticated user can only read/write objects under their own
{user_id}/ prefix
```

---

5. PRD User Story → Schema Validation Matrix

PRD User Story / Requirement	Schema Support
FR1: Parse PDF/DOCX within 5s	`resumes.raw_text` stores extracted text post-parse
FR2: AI structured JSON response persisted	`resumes.feedback_json` (jsonb)
FR3: Persist all analyses per user with timestamp	`resumes.user_id` + `created_at`, indexed
FR4: Dynamic interview question generation	`interview_sessions.questions_json` (jsonb, generated per session, not hardcoded)
FR5: Consistent scoring rubric via JSON schema	`interview_sessions.scores_json` + `overall_score` bounded check constraint
FR6: Dashboard visualizes ≥2 data types	`progress_snapshots` provides time-series `resume_score` (line chart) and `interview_score`/topics (radar/list)
5.1 Auth (email/Google)	Handled entirely by `auth.users` + Supabase Auth, no custom password storage
5.2 Resume Analyzer — ATS score, keyword gaps, section suggestions	`resumes.ats_score`, `feedback_json` (contains sections + missing_keywords per prompt schema in ARCHITECTURE.md)
5.3 Mock Interview — role-specific, resume-aware, scored	`interview_sessions.role`, `resume_id` FK, `questions_json`, `answers_json`, `scores_json`
5.4 Dashboard — history, weak topics, streak	`progress_snapshots` (history + weak_topics); streak computed at query time from `created_at` dates across `resumes`/`interview_sessions`
5.5 Settings/Profile — target role, delete account	`user_profiles.target_role`; account deletion cascades via `on delete cascade` on all FKs
Non-goal check: no payment/subscription tables	Confirmed — no `subscriptions`/`payments` table exists in v1, matching PRD non-goals

Result: Every functional requirement and user story in the PRD has a corresponding, traceable schema element. No orphan requirements, no unused tables.